

Vježba 1: Jednostavna forma za ponavljanje

Naziv projekta

08_React/06_Practice_SimpleControlledInput/

Cilj vježbe

U ovoj vježbi učenici će napraviti malu React aplikaciju s jednim kontroliranim inputom. Cilj je ponoviti osnovni obrazac:

```
value={state}  
onChange={handler}
```

Učenici trebaju razumjeti da input više ne čuva vrijednost samostalno. Vrijednost se nalazi u React stateu, a svaki unos korisnika ažurira taj state.

Scenarij

Zamislite da izrađujete malu aplikaciju za unos korisničkog imena. Korisnik treba upisati svoje ime, a aplikacija treba odmah prikazivati pozdravnu poruku ispod inputa.

Ako korisnik još ništa nije upisao, aplikacija treba prikazati neutralnu poruku.

Ako korisnik upiše ime, aplikacija treba prikazati personalizirani pozdrav.

Zadatak

1. Kreirajte novi React projekt

U terminalu pokrenite:

```
npm create vite@latest simple-controlled-input -- --template react
```

Uđite u projekt:

```
cd simple-controlled-input
```

Instalirajte pakete:

```
npm install
```

Pokrenite aplikaciju:

```
npm run dev
```

2. Očistite početni projekt

U datoteci `src/App.jsx` obrišite početni sadržaj i pripremite osnovnu strukturu:

```
import { useState } from "react";
import "./App.css";

function App() {
  return (
    <main className="app">
      <h1>Simple Name Form</h1>
    </main>
  );
}
```

```
export default App;
```

U datoteci `src/App.css` obrišite postojeći CSS.

3. Dodajte state za ime

Unutar `App` komponente dodajte state varijablu:

```
const [name, setName] = useState("");
```

Ova varijabla će čuvati trenutnu vrijednost inputa.

Početna vrijednost je prazan string jer input na početku nema nikakav tekst.

4. Dodajte kontrolirani input

U JSX dodajte `label` i `input`.

Input treba imati:

```
value={name}  
onChange={event => setName(event.target.value)}
```

Primjer:

```
<div className="form-group">  
  <label htmlFor="name">Your name:</label>  
  <input  
    id="name"  
    type="text"  
    value={name}  
    onChange={event => setName(event.target.value)}  
  />  
</div>
```

Ovdje je input kontroliran zato što njegovu vrijednost određuje React state.

5. Prikažite vrijednost u stvarnom vremenu

Ispod inputa prikažite trenutnu vrijednost:

```
<p>Current value: {name}</p>
```

Dok korisnik tipka, tekst se treba odmah mijenjati.

Ovo je najvažniji trenutak u vježbi: učenici trebaju vidjeti da input, state i prikaz na ekranu rade zajedno.

6. Dodajte uvjetnu poruku

Dodajte poruku koja ovisi o tome je li korisnik upisao ime.

Ako je `name` prazan string, prikažite:

Please enter your name.

Ako korisnik upiše ime, prikažite:

Hello, Ana!

Koristite ternarni operator:

```
<p className="message">
```

```
{name === "" ? "Please enter your name." : `Hello, ${name}!`}
</p>
```

7. Dodajte gumb za brisanje

Dodajte gumb koji vraća **name** na prazan string:

```
<button onClick={() => setName("")}>
  Clear
</button>
```

Kada korisnik klikne gumb, input se treba isprazniti.

Ovo pokazuje važnu ideju: budući da je input kontroliran, promjena statea mijenja i ono što input prikazuje.

Očekivano ponašanje aplikacije

Na početku aplikacija prikazuje:

```
Simple Name Form
Your name:
Please enter your name.
```

Kada korisnik upiše **Ana**, aplikacija prikazuje:

```
Current value: Ana
Hello, Ana!
```

Kada korisnik klikne **Clear**, input se isprazni i poruka se vrati na početno stanje.

Puno rješenje

src/App.jsx

```
import { useState } from "react";
import "./App.css";

function App() {
  const [name, setName] = useState("");
```

```

return (
  <main className="app">
    <h1>Simple Name Form</h1>

    <div className="form-group">
      <label htmlFor="name">Your name:</label>
      <input
        id="name"
        type="text"
        value={name}
        onChange={event => setName(event.target.value)}
      />
    </div>

    <p className="current-value">Current value: {name}</p>

    <p className="message">
      {name === "" ? "Please enter your name." : `Hello, ${name}!`}
    </p>

    <button onClick={() => setName("")}>
      Clear
    </button>
  </main>
);
}

export default App;

```

src/App.css

```

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #eef2f7;
}

.app {
  max-width: 480px;
  margin: 60px auto;
  padding: 32px;
  background-color: white;
  border: 1px solid #d8dee9;
  border-radius: 16px;
}

```

```
h1 {  
  margin-top: 0;  
  text-align: center;  
}
```

```
.form-group {  
  margin-bottom: 20px;  
}
```

```
label {  
  display: block;  
  margin-bottom: 8px;  
  font-weight: bold;  
}
```

```
input {  
  width: 100%;  
  box-sizing: border-box;  
  padding: 10px;  
  border: 1px solid #ccd3df;  
  border-radius: 8px;  
  font-size: 16px;  
}
```

```
.current-value {  
  color: #526070;  
}
```

```
.message {  
  font-size: 18px;  
  font-weight: bold;  
}
```

```
button {  
  padding: 10px 16px;  
  border: 1px solid #ccd3df;  
  border-radius: 8px;  
  background-color: #f7f9fc;  
  cursor: pointer;  
}
```

```
button:hover {  
  background-color: #e9eef6;  
}
```